Data Structures and Algorithms

1. Introduction to Binary Trees

A binary tree is a hierarchical data structure in which each node has at most two children, referred to as the left child and the right child. Binary trees are fundamental structures used in many algorithms and data organization schemes.

Properties of binary trees include:
- The maximum number of nodes at level k is 2^k
- A tree with n nodes has exactly n-1 edges
- The height of a complete binary tree with n nodes is log2(n)

Binary trees support efficient operations for searching, insertion, and deletion when properly balanced. Common applications include expression parsing, decision trees, and hierarchical data representation.

2. Hash Tables and Collision Resolution

Hash tables provide average-case O(1) time complexity for insertion, deletion, and lookup operations. A hash function maps keys to array indices, enabling direct access to stored values.

Common collision resolution strategies:
- Chaining: Store colliding elements in linked lists
- Open addressing: Find alternative slots using probing sequences
- Double hashing: Use a secondary hash function for probing

The load factor (number of elements divided by table size) significantly impacts performance. A load factor between 0.5 and 0.75 typically provides good performance while maintaining reasonable space efficiency.

Hash tables are widely used in database indexing, caching systems, and symbol tables in compilers. Modern implementations often use techniques like Robin Hood hashing or Cuckoo hashing to further improve performance and reduce worst-case behavior.